



Lecture_1

Windows Programming_2

Dr. Sara Mohamed

The HScrollBar and VScrollBar Controls

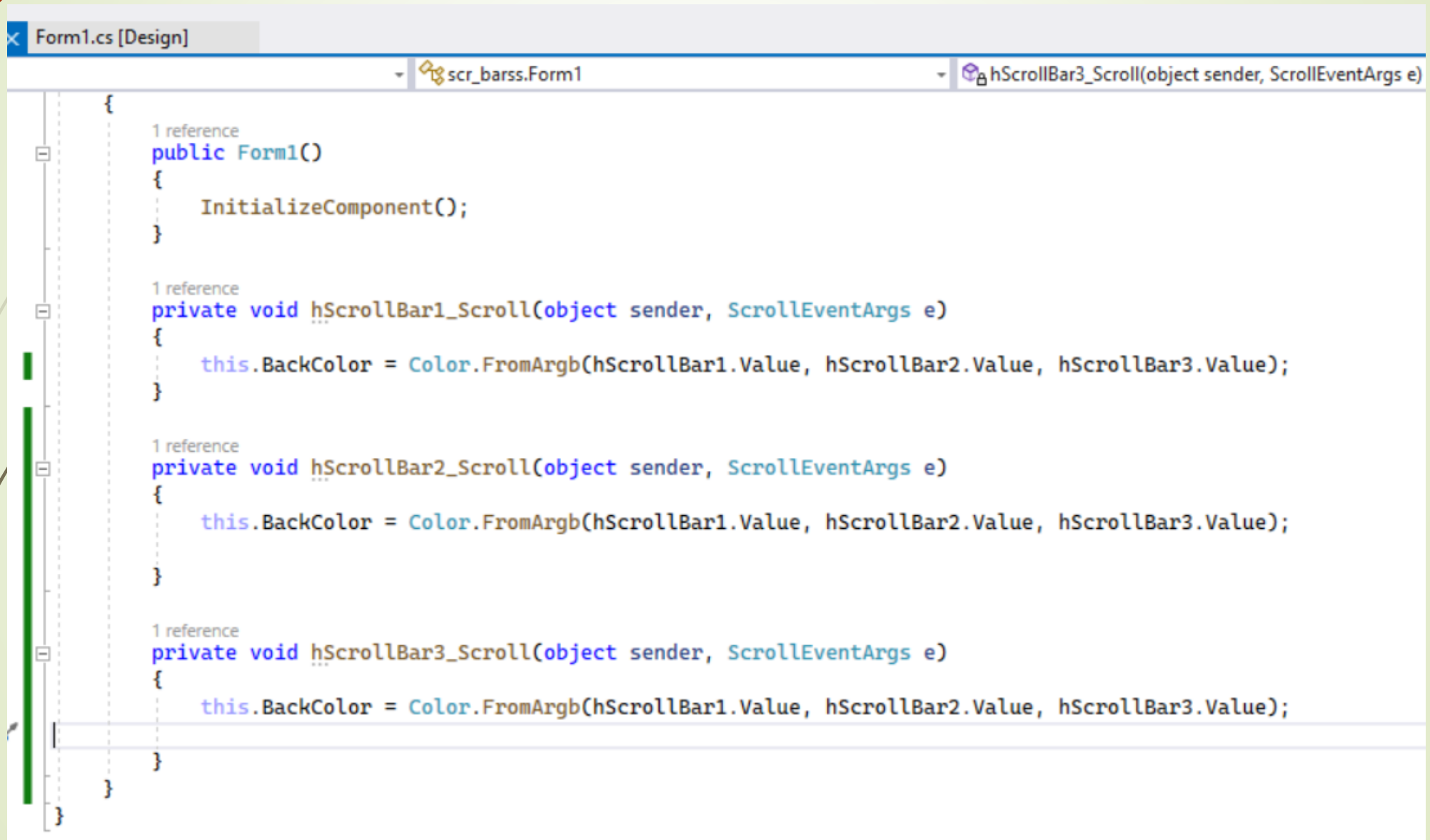
The **HScrollBar** and **VScrollBar** controls display horizontal and vertical scrollbars, respectively. The HScrollBar and VScrollBar classes inherit their properties and other members from the **ScrollBar** class, which provides the basic scrolling functionality.

Usually, controls such as Panel, TextBox, and ComboBox include their own scrollbars. But some controls, such as **PictureBox**, do not have built-in scrollbars. You can use HScrollBar and VScrollBar to associate scrollbars with such controls

IMPORTANT MEMBERS OF THE ScrollBar CLASS

<i>Member</i>	<i>Type</i>	<i>Description</i>
LargeChange	Property	Indicates the number by which the Value property changes when the scroll box is moved a large distance.
Maximum	Property	Specifies the upper bound of the scrollbar's range.
Minimum	Property	Specifies the lower bound of the scrollbar's range.
Scroll	Event	Occurs when the scroll box is moved by a keyboard or mouse action. Scroll is the default event for the ScrollBar class.
SmallChange	Property	Indicates the number by which the Value property changes when the scroll box is moved a small distance.
Value	Property	Represents the current position of the control.
ValueChanged	Event	Occurs when the Value property changes either via the Scroll event or programmatically.

HScrollBar Control example_1



```
Form1.cs [Design]
scr_barss.Form1
hScrollBar3_Scroll(object sender, ScrollEventArgs e)

{
    1 reference
    public Form1()
    {
        InitializeComponent();
    }

    1 reference
    private void hScrollBar1_Scroll(object sender, ScrollEventArgs e)
    {
        this.BackColor = Color.FromArgb(hScrollBar1.Value, hScrollBar2.Value, hScrollBar3.Value);
    }

    1 reference
    private void hScrollBar2_Scroll(object sender, ScrollEventArgs e)
    {
        this.BackColor = Color.FromArgb(hScrollBar1.Value, hScrollBar2.Value, hScrollBar3.Value);
    }

    1 reference
    private void hScrollBar3_Scroll(object sender, ScrollEventArgs e)
    {
        this.BackColor = Color.FromArgb(hScrollBar1.Value, hScrollBar2.Value, hScrollBar3.Value);
    }
}
```

Output

Form1

< >

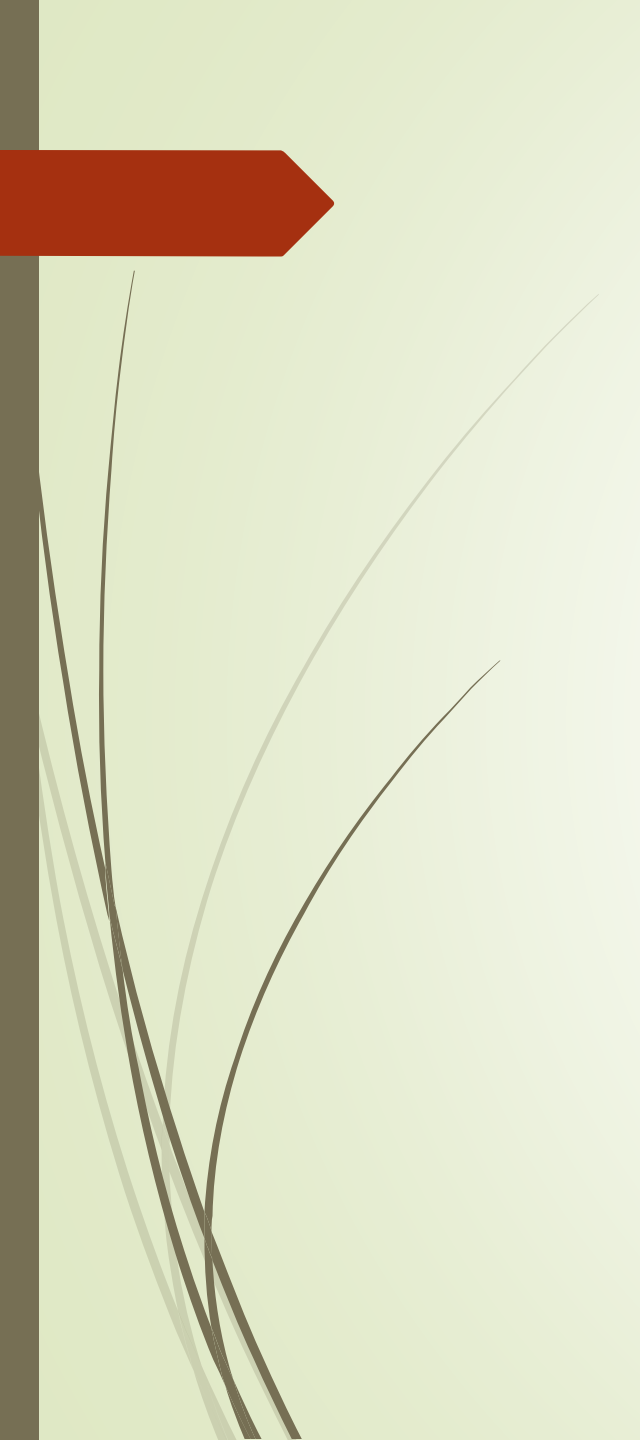
Red

< >

Green

< >

Blue



Working with dialogs



DIALOG BOXES

- A dialog box is used to prompt the user for input. The application can then use the user input for its own processing. You can either use one of the existing dialog box components provided by the Windows Forms library or you can create a dialog box to meet your custom application requirements.

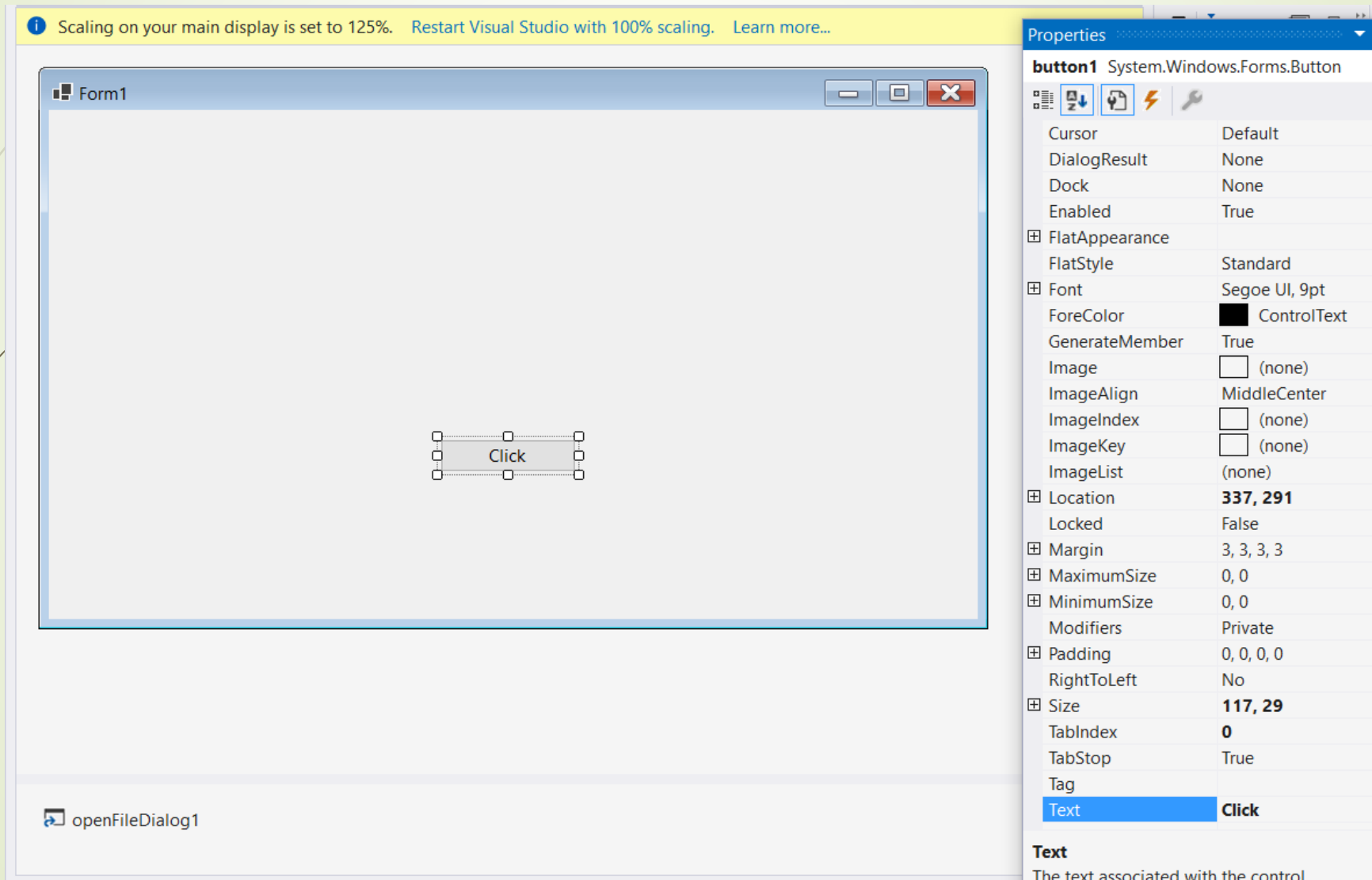
Common Dialog Boxes

The Windows Forms library provides the following dialog box classes that are ready to use in Windows applications:

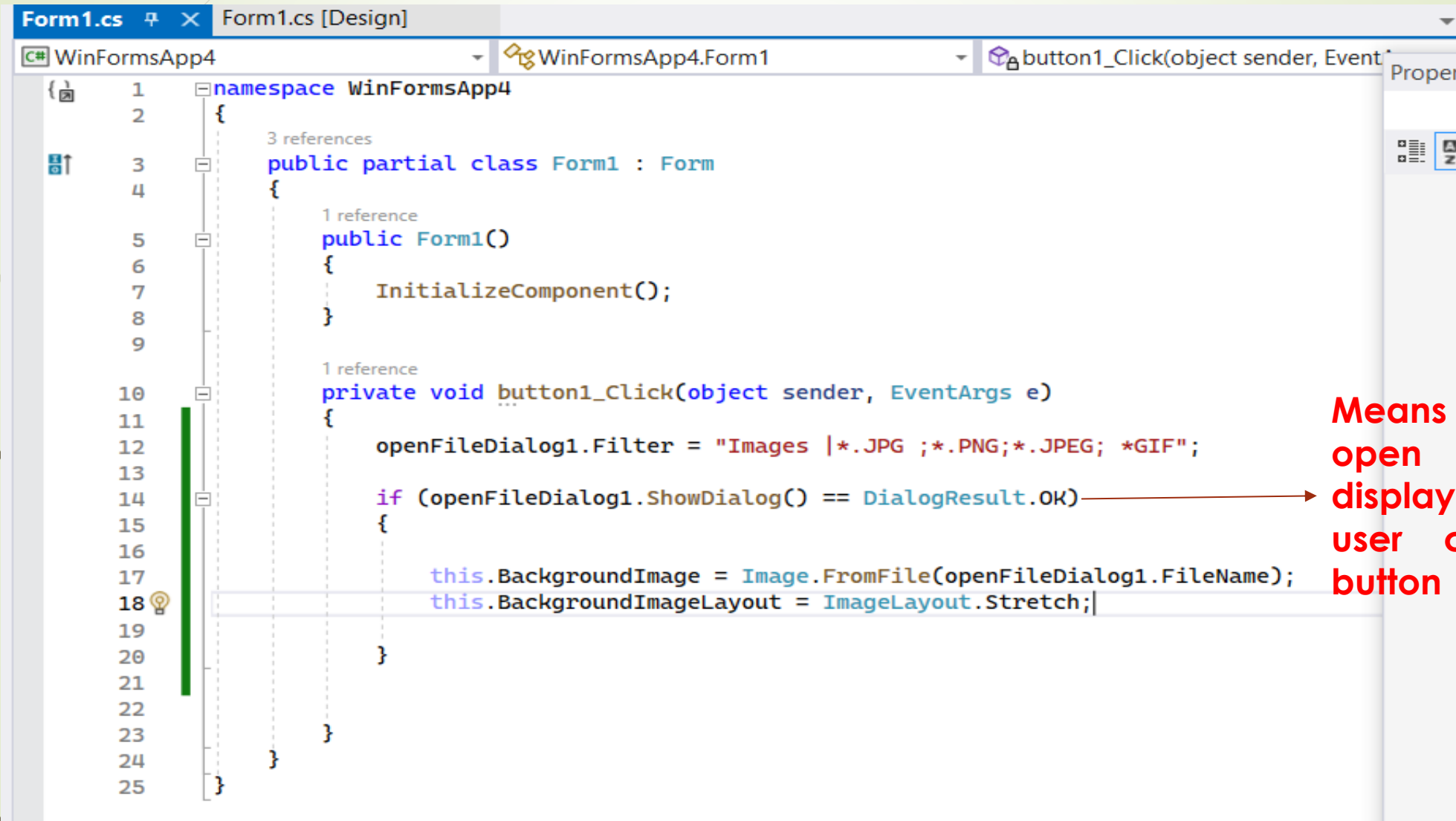
- **ColorDialog**: Displays a list of colors and returns a property that contains the color selected by user.
- **FontDialog**: Displays a dialog box that allows the user to select a font and other text properties, such as size, style, and special effects.
- **OpenFileDialog**: Allows the user to browse files and folders on his or her computer and select one or more files.
- **PageSetupDialog**: Allows the user to select various settings related to page layout.
- **PrintDialog**: Allows the user to select various print-related options and sends specified documents to selected printers.
- **PrintPreviewDialog**: Allows the user to preview a file before printing.
- **SaveFileDialog**: Allows the user to browse the files and folders on his or her computer and select files that need to be saved

- 
- These classes are also referred to as Windows Forms dialog components. These dialog boxes provide the same functionality found in several of the common dialog boxes that are used by the Windows operating system. Each of these dialog box classes is derived from the **CommonDialog** class, which provides the basic functionality for displaying a dialog box.
 - The dialog box classes provide a method named **ShowDialog()** that presents a dialog box to the user. Each of the dialog box classes has a set of properties that store data that is relevant to the particular dialog box.

OpenFileDialog Example

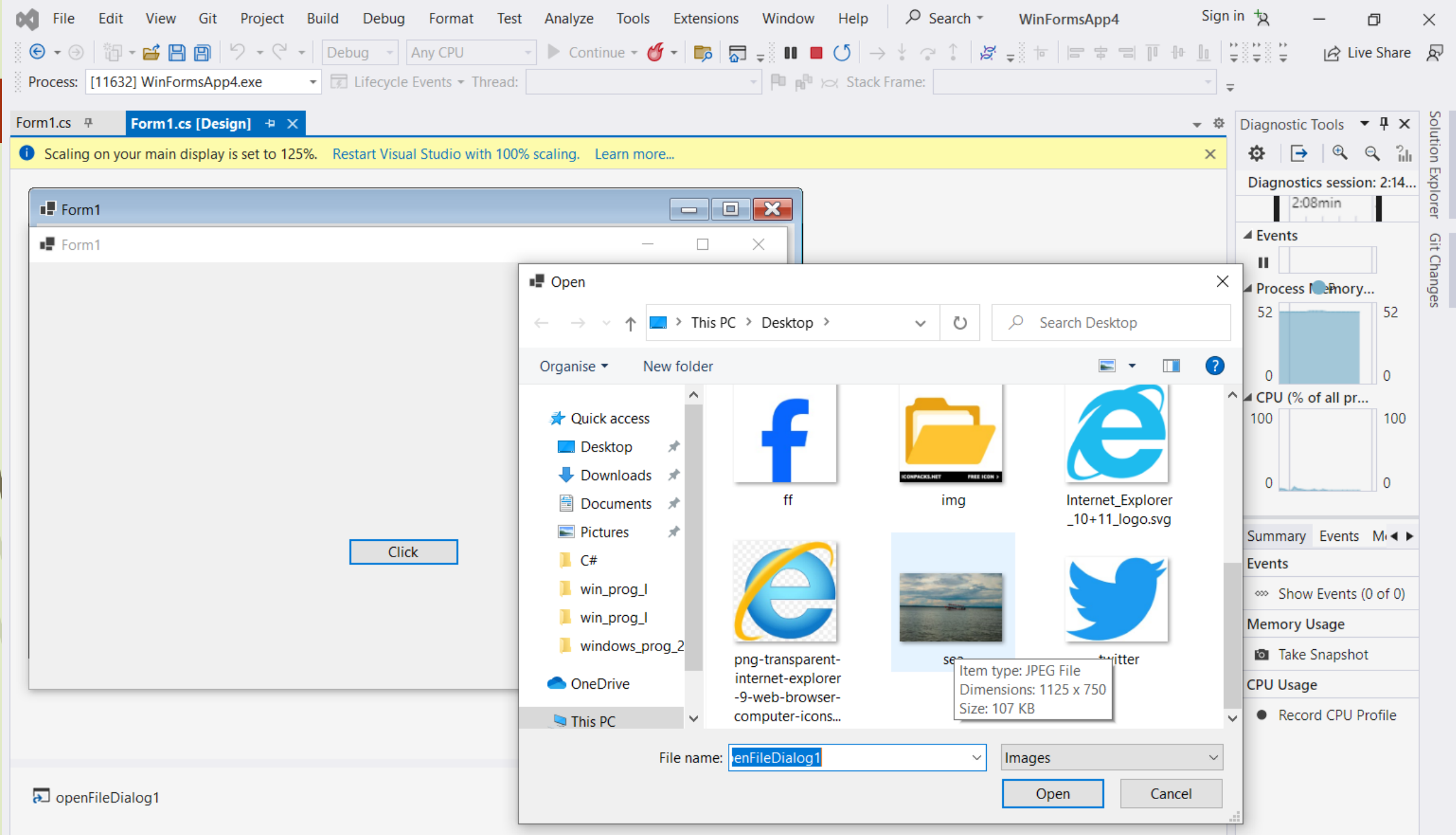


Get an image file then make it as a background for the form

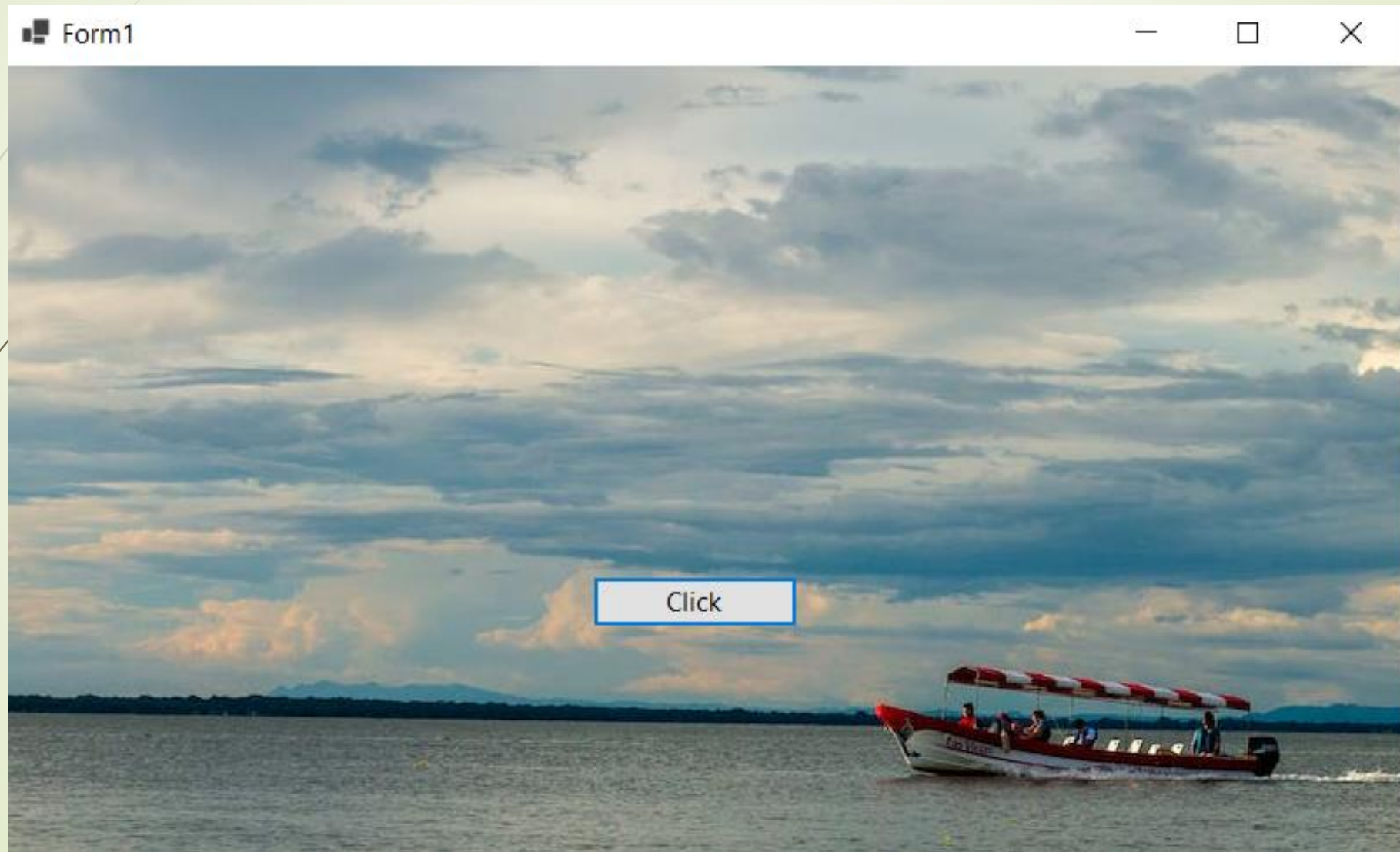


```
1 namespace WinFormsApp4
2 {
3     3 references
4     public partial class Form1 : Form
5     {
6         1 reference
7         public Form1()
8         {
9             InitializeComponent();
10
11         1 reference
12         private void button1_Click(object sender, EventArgs e)
13         {
14             openFileDialog1.Filter = "Images |*.JPG ;*.PNG;*.JPEG; *GIF";
15
16             if (openFileDialog1.ShowDialog() == DialogResult.OK)
17             {
18                 this.BackgroundImage = Image.FromFile(openFileDialog1.FileName);
19                 this.BackgroundImageLayout = ImageLayout.Stretch;
20             }
21         }
22     }
23 }
24
25 }
```

Means that the open dialog is displayed then user clicked a button

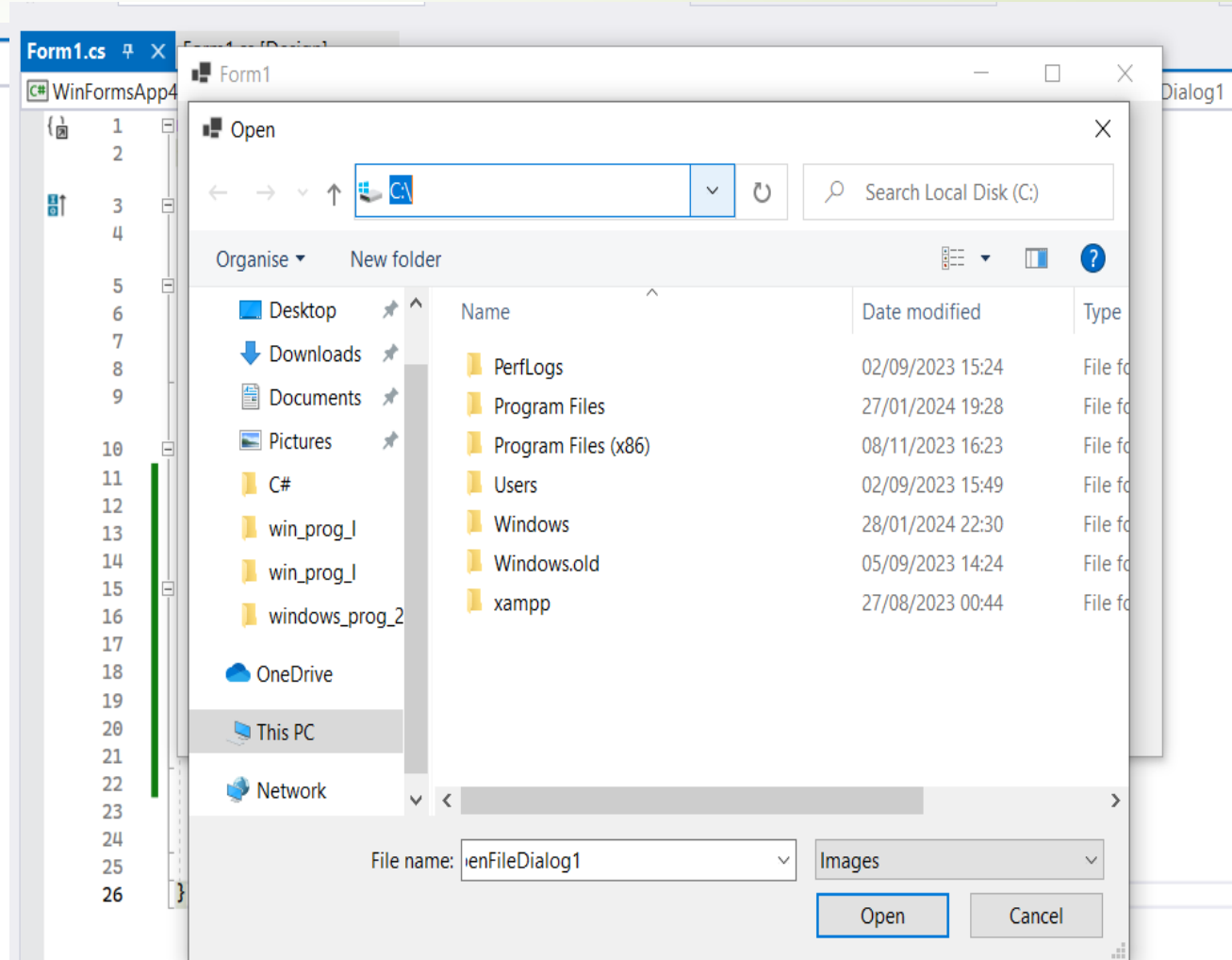


Output



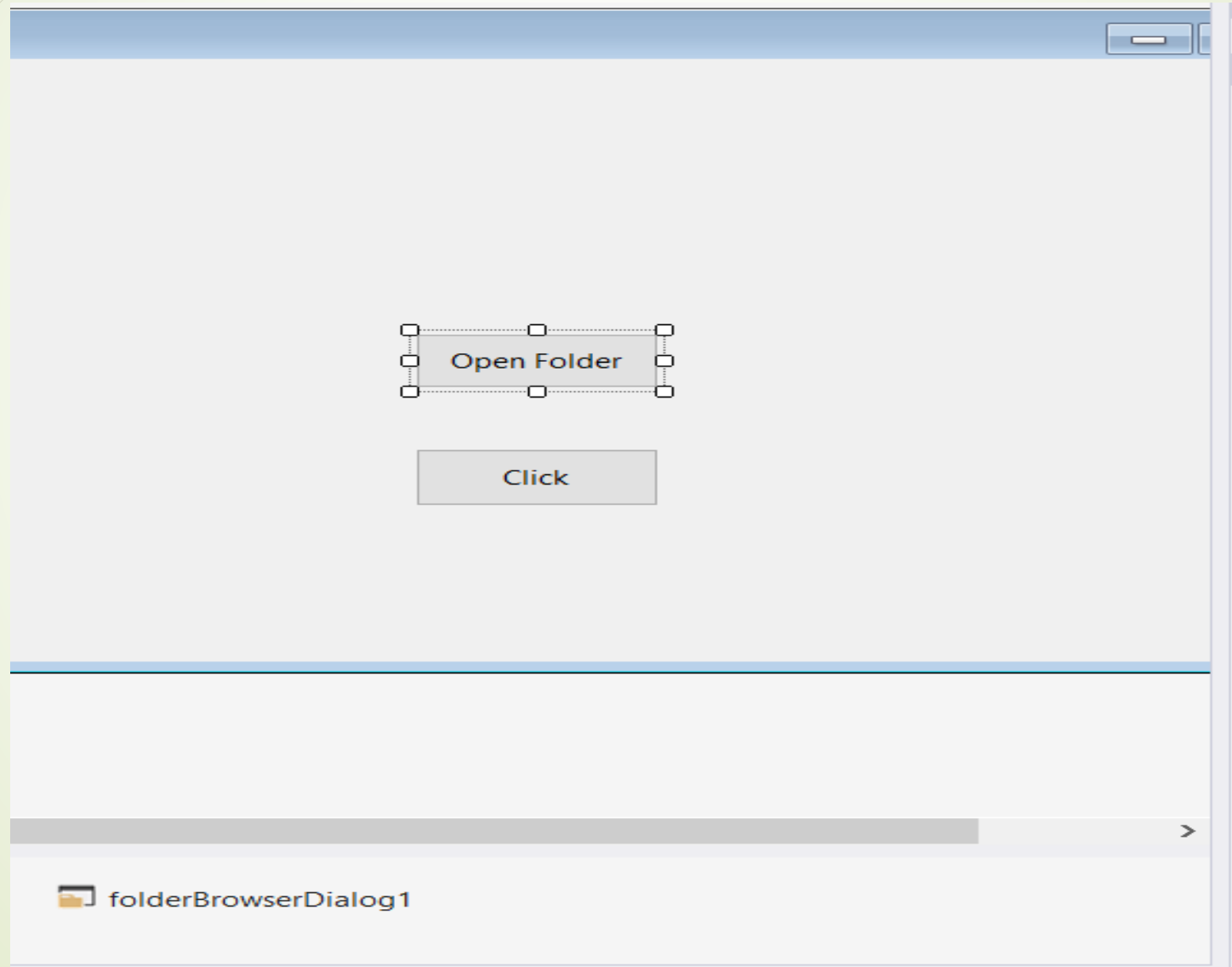
You can specify the directory which opens first

```
WinFormsApp4 | WinFormsApp4.Form1 | openFileDialog1
1 namespace WinFormsApp4
2 {
3     3 references
4     public partial class Form1 : Form
5     {
6         1 reference
7         public Form1()
8         {
9             InitializeComponent();
10
11         1 reference
12         private void button1_Click(object sender, EventArgs e)
13         {
14             openFileDialog1.Filter = "Images |*.JPG ;*.PNG;*.JPEG; *GIF";
15             openFileDialog1.InitialDirectory = "C:\\\\";
16
17             if (openFileDialog1.ShowDialog() == DialogResult.OK)
18             {
19                 this.BackgroundImage = Image.FromFile(openFileDialog1.FileName);
20                 this.BackgroundImageLayout = ImageLayout.Stretch;
21             }
22         }
23     }
24 }
25
26 }
```

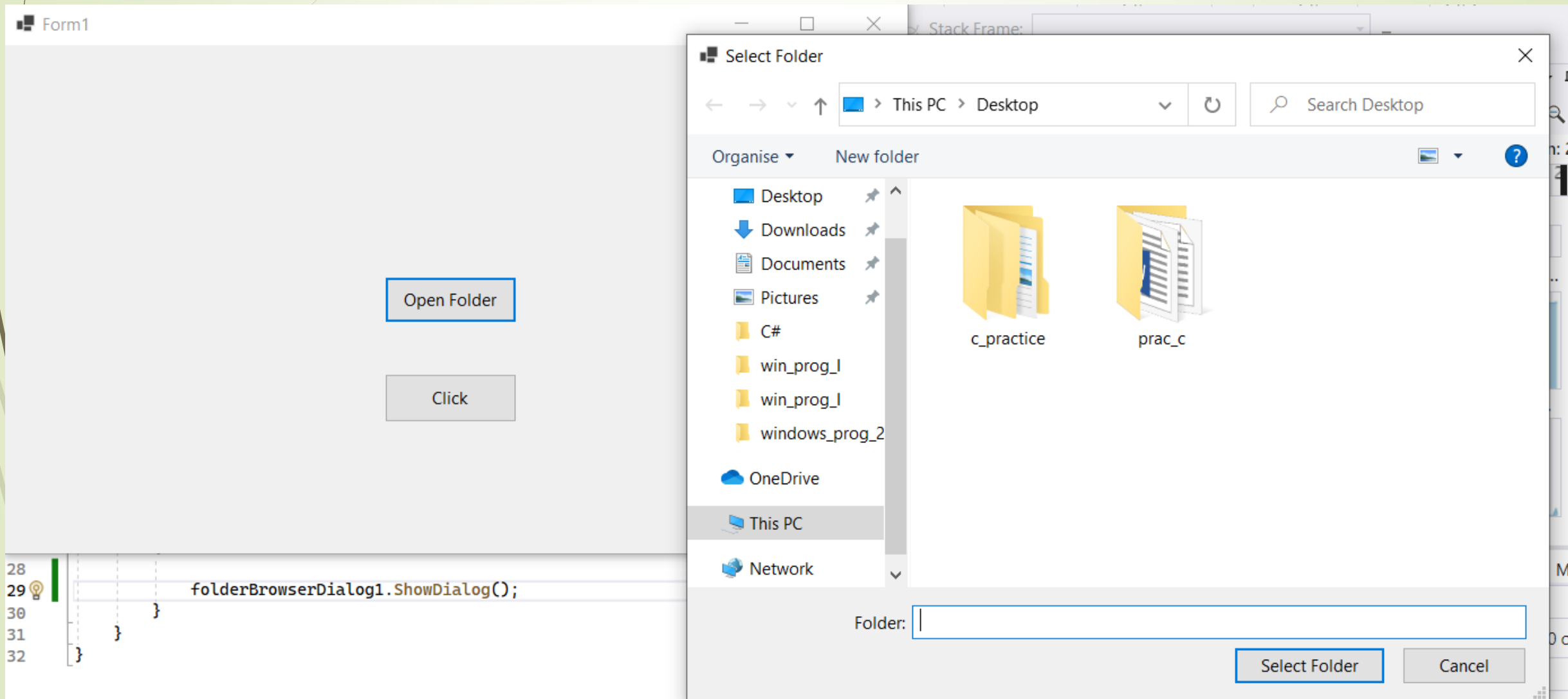


folderBrowserDialog Example

→ This control prompts the user to select a folder



Double click on the Open Folder Button and type the following code



You can know the selected folder path

1 reference

```
private void button2_Click(object sender, EventArgs e)
{
    if (folderBrowserDialog1.ShowDialog() == DialogResult.OK)
    {
        MessageBox.Show("Path of folder is:" + folderBrowserDialog1.SelectedPath);
    }
}
```

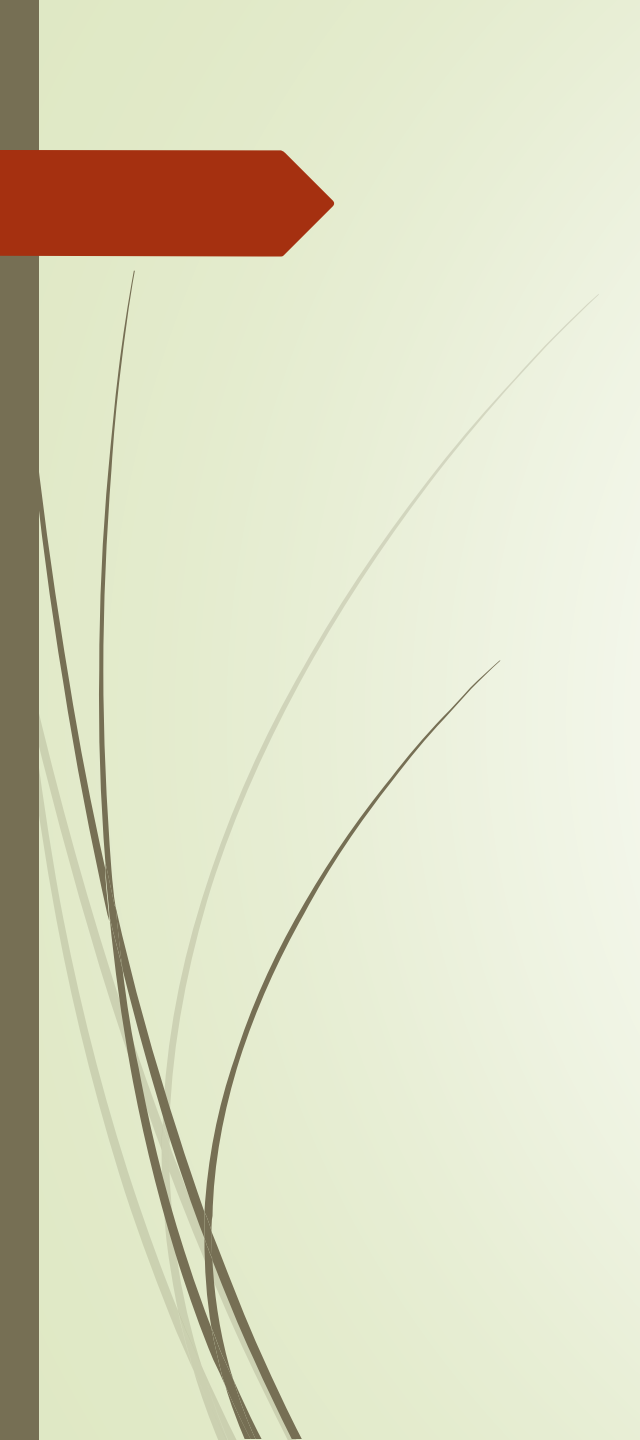
Form1

Open Folder

Click

Path of folder is:D:\prog_source\codeblocks

OK



Example

Common Dialog Boxes example

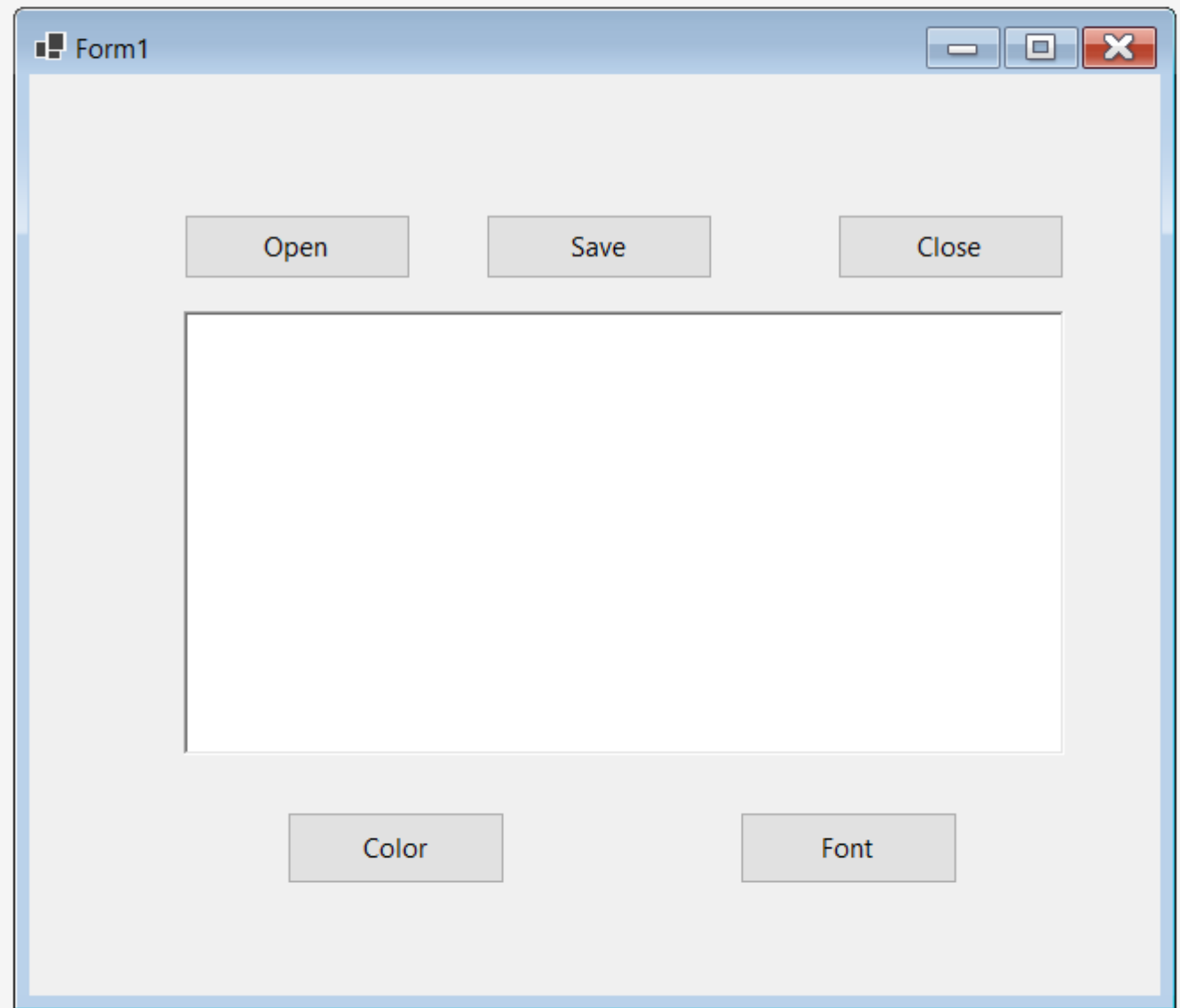
- Add a Windows Form to existing project
- Place five Button controls on the form. Name them btnOpen, btnSave, btnClose, btnColor, and btnFont and change their Text properties to &Open..., &Save..., Close&..., &Color..., and &Font..., respectively
- Place a **RichTextBox** control on the form and name it rtbText. Arrange all the controls as shown in the following figure :

The screenshot shows a Windows Form titled "Form1" with a standard Windows XP-style title bar (minimize, maximize, close buttons). The form contains five buttons and one RichTextBox control. The buttons are arranged as follows: "Open", "Save", and "Close" are in a horizontal row at the top. Below them is a large, empty RichTextBox control. At the bottom, "Color" and "Font" buttons are in a horizontal row. All buttons are light gray with black text.

➡ Drag and drop the following components from the toolbox to the form:

➡ OpenFileDialog, SaveFileDialog, ColorDialog, and FontDialog.

Because these are components, they are not added to the form, but they appear on the component tray in the lower area of the form:



The screenshot shows a standard Windows application window titled "Form1". Inside the window, there is a light gray background. At the top, there are three buttons labeled "Open", "Save", and "Close" arranged horizontally. In the center of the window is a large, empty white rectangular area. At the bottom, there are two buttons labeled "Color" and "Font" arranged horizontally. The window has a standard Windows title bar with minimize, maximize, and close buttons.

 openFileDialog1

 saveFileDialog1

 colorDialog1

 fontDialog1

Double-click the Open button to attach an event handler to the **Click** event. Add the following code to the event handler:

```
}  
  
1 reference  
private void btnOpen_Click(object sender, EventArgs e)  
{  
    //Allow to select only *.rtf files  
    openFileDialog1.Filter = "Rich Text Files (*.rtf)|*.rtf";  
    if (openFileDialog1.ShowDialog() == DialogResult.OK)  
    {  
        //Load the file contents in the RichTextBox  
        rtbText.LoadFile(openFileDialog1.FileName, RichTextBoxStreamType.RichText);  
    }  
}
```




Home

New

Open

Share

Info

Save

Save As

Print

Export

Close

Account

Options

Save As

Save As

« Wind... » Network Short...

Search Network Shortcuts

Organize New folder

3D Objects

Desktop

Documents

Downloads

Music

Pictures

Videos

Local Disk (C:)

New Volume (D:)

Name

Date modified

Type

No items match your search.

File name: Doc2.odt

Save as type: OpenDocument Text (*.odt)

Authors:

Word Document (*.docx)

Word Macro-Enabled Document (*.docm)

Word 97-2003 Document (*.doc)

Word Template (*.dotx)

Word Macro-Enabled Template (*.dotm)

Word 97-2003 Template (*.dot)

PDF (*.pdf)

XPS Document (*.xps)

Single File Web Page (*.mht;*.mhtml)

Web Page (*.htm;*.html)

Web Page, Filtered (*.htm;*.html)

Rich Text Format (*.rtf)

Plain Text (*.txt)

Word XML Document (*.xml)

Word 2003 XML Document (*.xml)

Strict Open XML Document (*.docx)

OpenDocument Text (*.odt)

Save

Date modified

plates

24/12/2022 20:59

21/09/2023 12:12

21/09/2023 12:12

ement Studio

04/04/2024 12:35

02/12/2025 00:13

04/04/2024 17:30

Open

Save

Close

التجارة الإلكترونية هي عملية بيع وشراء
المنتجات بالوسائل الإلكترونية مثل تطبيقات
الهاتف المحمول والإنترنت

Color

Font

Open



> This PC > Desktop



Search Desktop

Organise

New folder

Desktop
Downloads
Documents
Pictures
C#
win_prog_I
win_prog_I
windows_prog_2

OneDrive

This PC

Name

Date modified

Type

c_practice

26/12/2023 13:46

File fo

prac_c

26/12/2023 13:44

File fo

Time Management

09/02/2024 14:26

Rich T

ساره

09/02/2024 14:30

Rich T

File name: phamed\Desktop\Time Management

Rich Text Files (*.rtf)

Open

Cancel

Add the following code to handle the Click event of the Save button:

```
1 reference
private void btnSave_Click(object sender, EventArgs e)
{
    //Default choice to save file is *.rtf
    //but user can select
    //All Files to save with other extension
    saveFileDialog1.Filter = "Rich Text Files(*.rtf) | *.rtf | All Files(*.*) | *.*";

    if (saveFileDialog1.ShowDialog() == DialogResult.OK)
    {
        //Save the RichText content to a file
        rtbText.SaveFile(saveFileDialog1.FileName, RichTextBoxStreamType.RichText);
    }
}
```

Add the following code to handle the Click event of the Close button:

```
1 reference
private void btnClose_Click(object sender, EventArgs e)
{
    //close the form
    this.Close();
}
```

Add the following code to handle the Click event of the Color and Font buttons:

You select a color from the dialog color box

You also select font size, style and size from the font dialog box

```
}  
  
1 reference  
private void button4_Click(object sender, EventArgs e)  
{  
    if (colorDialog1.ShowDialog() == DialogResult.OK)  
    {  
        //Change the color of selected text  
        //If no text selected, change the active color  
        rtbText.SelectionColor = colorDialog1.Color;  
    }  
}
```

```
1 reference  
private void button5_Click(object sender, EventArgs e)  
{  
    if (fontDialog1.ShowDialog() == DialogResult.OK)  
    {  
        //Change the font of selected text  
        //If no text selected, change the active font  
        rtbText.SelectionFont = fontDialog1.Font;  
    }  
}
```

What is Rich Text Format (RTF)?

- Rich Text Format (often abbreviated RTF) is a proprietary document file format with published specification developed by Microsoft Corporation from 1987 until 2008 for cross-platform document interchange with Microsoft products.



Open

Save

Close

التجارة الإلكترونية هي عملية بيع وشراء
المنتجات بالوسائل الإلكترونية مثل تطبيقات
الهاتف المحمول والإنترنت

Color

Font

Font



Font

Segoe UI Historic

Segoe UI Historic

Segoe UI Symbol

SHOWCARD GOTHIC

Simple Indust Outline

Simple Indust Shaded

Simple Jut Out

Font style:

Bold Oblique

Regular

Oblique

Bold

Bold Oblique

Size:

18

18

20

22

24

26

28

36

OK

Cancel

Effects

☐ Strikeout

☐ Underline

Sample

AaBbYyZz

Script

Western

Form1

Open Save Close

التجارة الإلكترونية هي عملية بيع وشراء
المنتجات بالوسائل الإلكترونية مثل تطبيقات
الهاتف المحمول والإنترنت

Color Font

Colour

Basic colours:

Custom colours:

Define Custom Colours >>

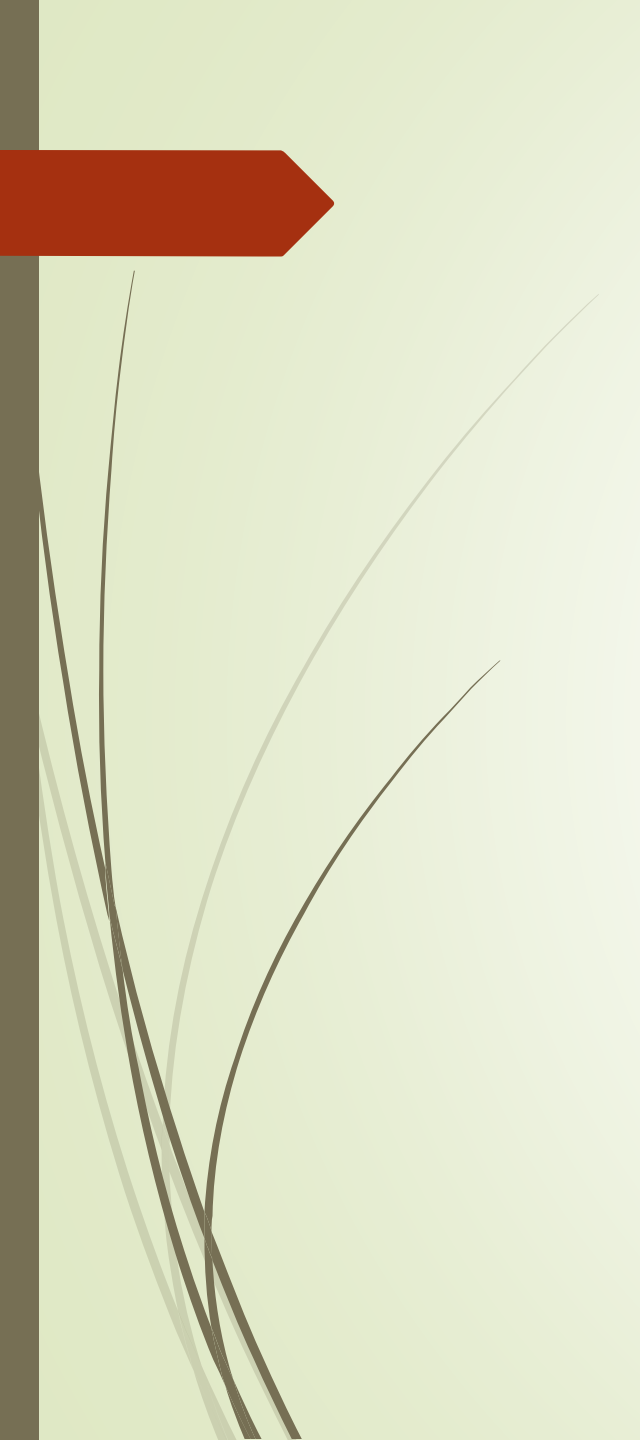
OK Cancel

The Filter property

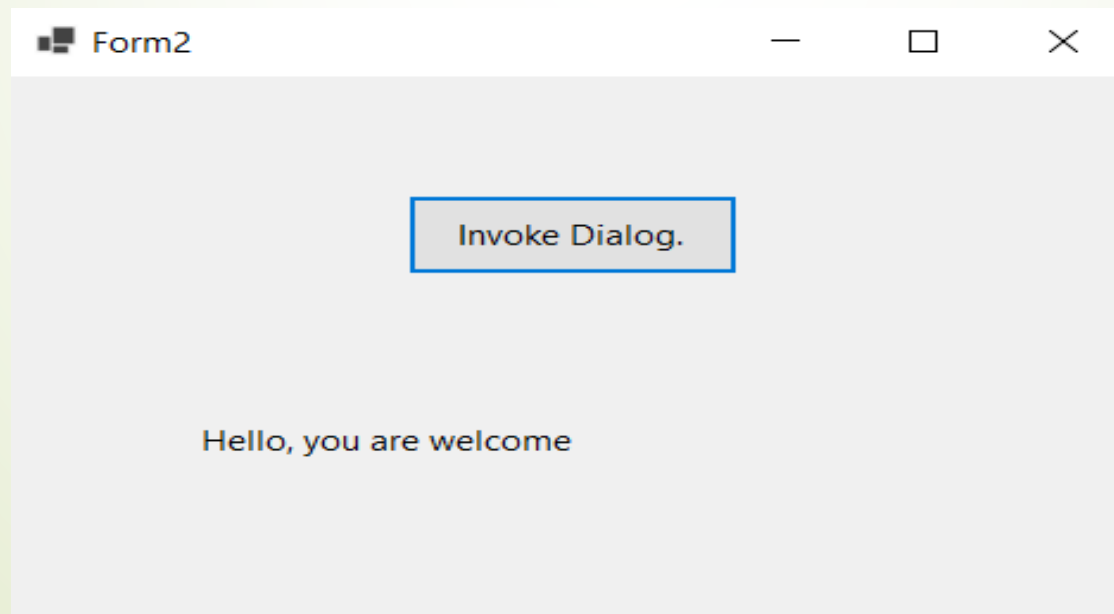
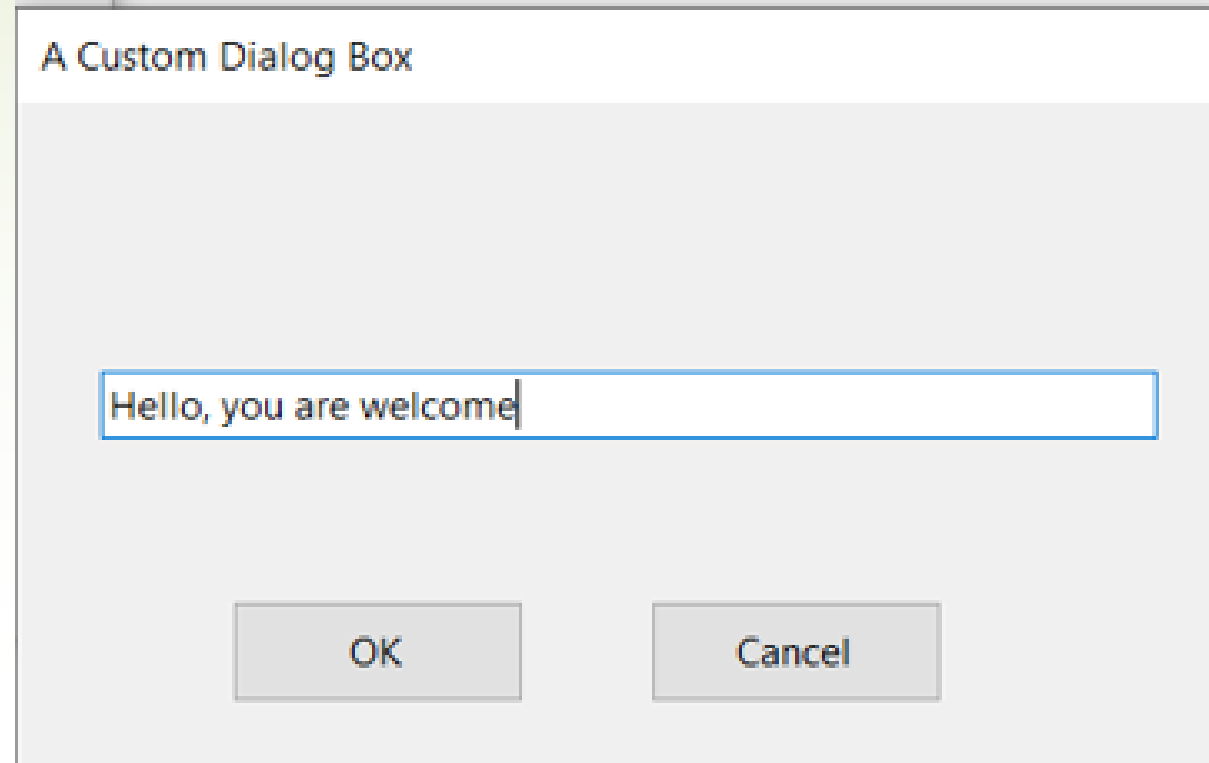
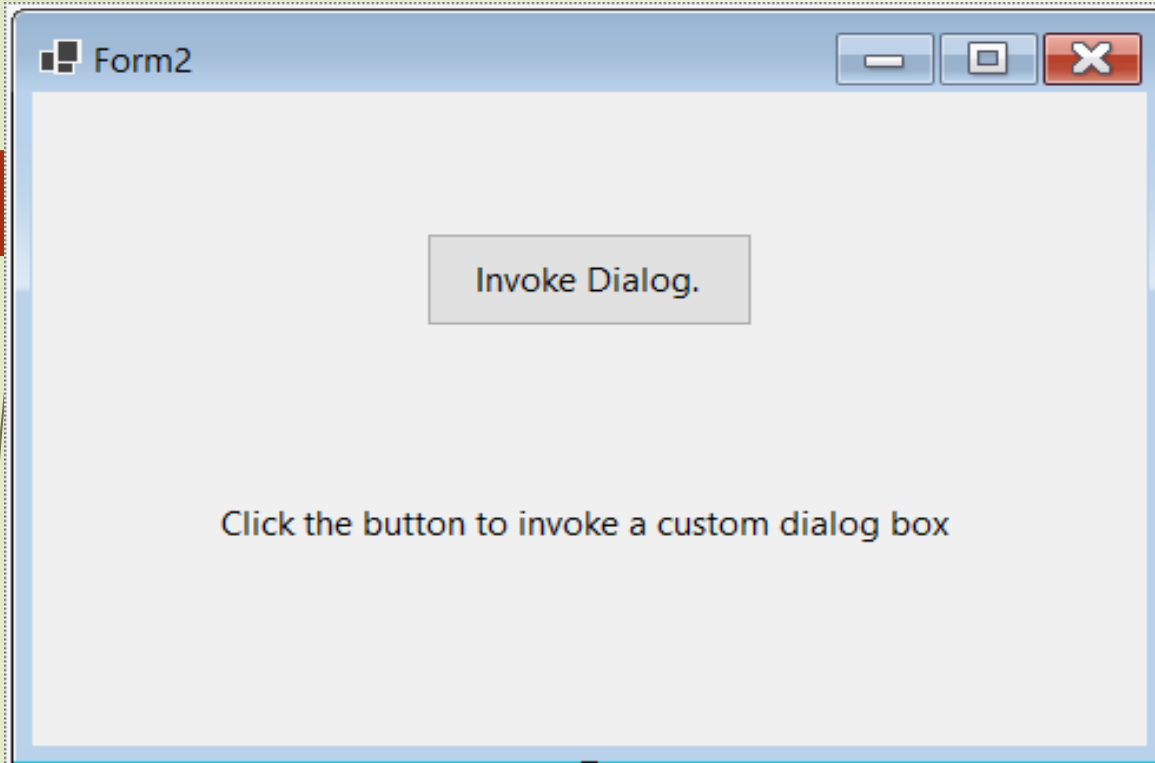
- The **Filter** property of the OpenFileDialog and SaveFileDialog components specifies the choices that appear in the **Files of Type** drop-down list boxes of these dialog boxes. You can use this property to **filter the type of files** that the user can select from the dialog box.
- To open the word files, you can use :
- `openFileDialog1.Filter = "Word Files (*.docx;*.doc)|*.docx;*.doc";`

Creating a Custom Dialog Box

- If you need to create dialog boxes other than those already provided by the Windows Forms library, you can do so by creating a form and setting it up to behave as a dialog box. You can make the dialog box as rich as your requirements dictate by adding various controls to it.



Example





Design the **frmDialog** form

- 
- Add a Windows form Name the form **frmDialog**
 - Set the **ControlBox** property of the form to false, set **FormBorderStyle** to **FixedDialog**, set **ShowIntaskBar** to **False**, set **StartPosition** to **CenterParent**, and set **Text** to **A Custom Dialog Box**.
 - Place two Button controls and a TextBox control on the form, set the Name property of the Button controls to **btnOK** and **btnCancel** and change their Text properties to **&OK** and **&Cancel**, respectively. Change the TextBox control's Name property to **txtDialogText**.



A Custom Dialog Box



OK

Cancel

Change the form's constructor code to the following

```
namespace winFormsApps
{
    4 references
    public partial class frmDialog : Form
    {
        1 reference
        public frmDialog()
        {
            InitializeComponent();
            //make the default value for the btnok dialog result to ok

            btnOK.DialogResult = System.Windows.Forms.DialogResult.OK;
            //make the default value for the btncancel dialog result to cancel
            btnCancel.DialogResult = DialogResult.Cancel;
        }
    }
}
```

```
3 references
public partial class frmDialog : Form
{
    1 reference
    public frmDialog()
    {
        InitializeComponent();

        btnOK.DialogResult = System.Windows.Forms.DialogResult.OK;
    }
}
```

enum System.Windows.Forms.DialogResult
Specifies identifiers to indicate the return value of a dialog box.

DialogResult enum in
System.Windows.Forms
namespace

```
Form1.cs*  Form1.cs [Design]*
WinFormsApp5  WinFormsApp5.frmDialog  frmDialog0

1 namespace WinFormsApp5
2 {
3     3 references
4     public partial class frmDialog : Form
5     {
6         1 reference
7         public frmDialog()
8         {
9             InitializeComponent();
10
11             btnOK.DialogResult = System.Windows.Forms.DialogResult.OK;
12
13         }
14
15     }
16
17     1 reference
18     private void btnOK_Click(object sender, EventArgs e)
19     {
20
21     }
```

DialogResult.Cancel = 2
The dialog box return value is Cancel (usually sent from a button labeled Cancel).
★ IntelliCode suggestion based on this context

- ★ Cancel
- ★ Continue
- ★ Yes
- ★ OK
- ★ Abort
- Abort
- Cancel
- Continue
- Ignore

```

1 namespace WinFormsApp7
2 {
3     3 references
4     public partial class Form1 : Form
5     {
6         1 reference
7         public Form1()
8         {
9             InitializeComponent();
10
11         1 reference
12         private void button1_Click(object sender, EventArgs e)
13         {
14             button1.DialogResult
15
16         }
17     }
18 }
    
```

- DialogResult
- DisplayRectangle
- Dispose
- Disposed
- Disposing
- IsDisposed

DialogResult Button.DialogResult { get; set; }
Gets or sets a value that is returned to the parent form when the button is clicked.

- Dependencies
- Form1.cs
 - Form1.Designer.cs
 - Form1.resx
- Program.cs

Add the following code just after the constructor to create a property that holds the text entered by the user

```
namespace WinFormsApp5
{
    4 references
    public partial class frmDialog : Form
    {
        1 reference
        public frmDialog()
        {
            InitializeComponent();
            //make the default value for the btnok dialog result to ok

            btnOK.DialogResult = System.Windows.Forms.DialogResult.OK;
            //make the default value for the btncancel dialog result to cancel
            btnCancel.DialogResult = DialogResult.Cancel;
        }

        private string message;
        //Stores the message entered by user
        2 references
        public string Message
        {
            get
            {
                return message;
            }
            set
            {
                message = value;
            }
        }
    }
}
```

Add an event handler for the Click event of the OK button and add the following code :

sApp5 WinFormsApp5.frmDialog btnCancel

```
4 references
public partial class frmDialog : Form
{
    1 reference
    public frmDialog()
    {
        InitializeComponent();
        //make the default value for the btnok dialog result to ok
        btnOK.DialogResult = System.Windows.Forms.DialogResult.OK;
        //make the default value for the btncancel dialog result to cancel
        btnCancel.DialogResult = DialogResult.Cancel;
    }

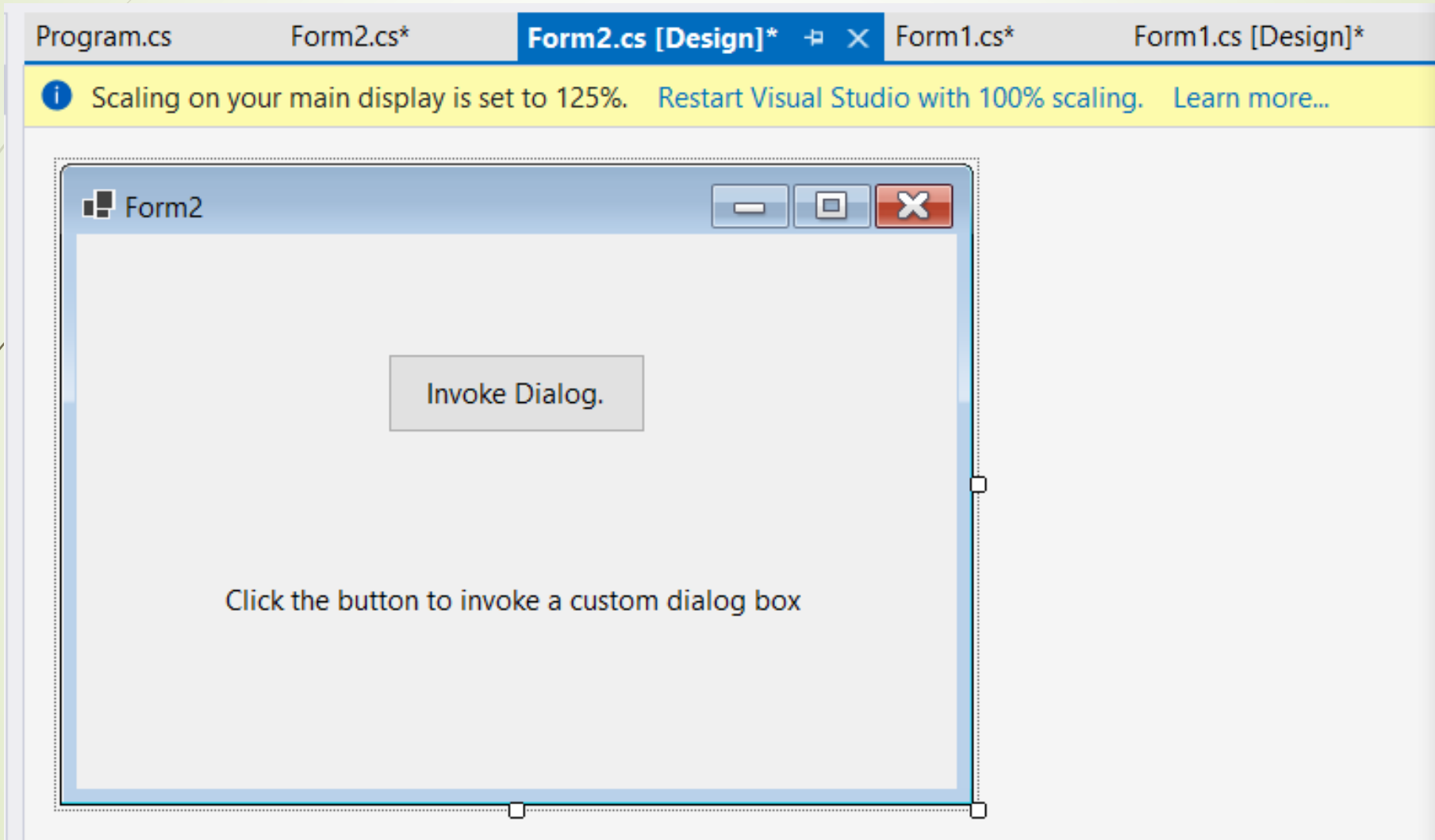
    private string message;
    //Stores the message entered by user
    2 references
    public string Message
    {
        get
        {
            return message;
        }
        set
        {
            message = value;
        }
    }

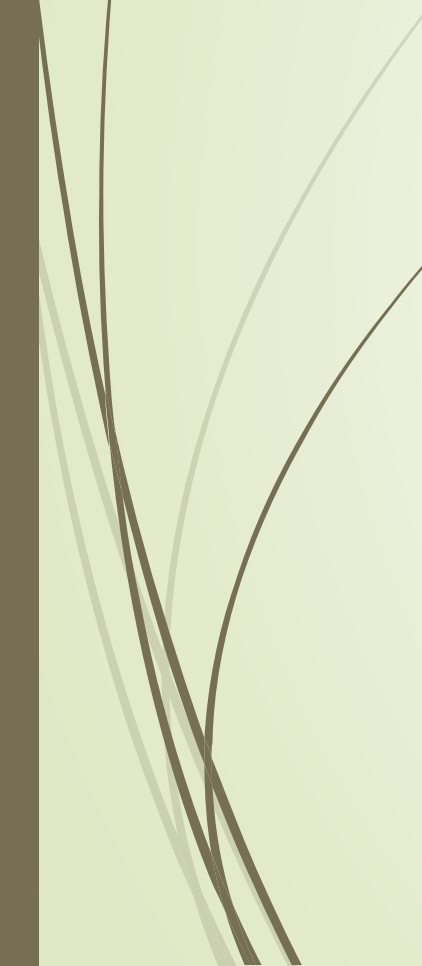
    1 reference
    private void btnOK_Click(object sender, EventArgs e)
    {
        this.Message = this.txtDialogText.Text;
    }
}
```



The diagram shows a rectangular dialog box with a dashed border. Inside, there is a text input field. An arrow points from the text 'this.txtDialogText.Text' in the code to this text input field.

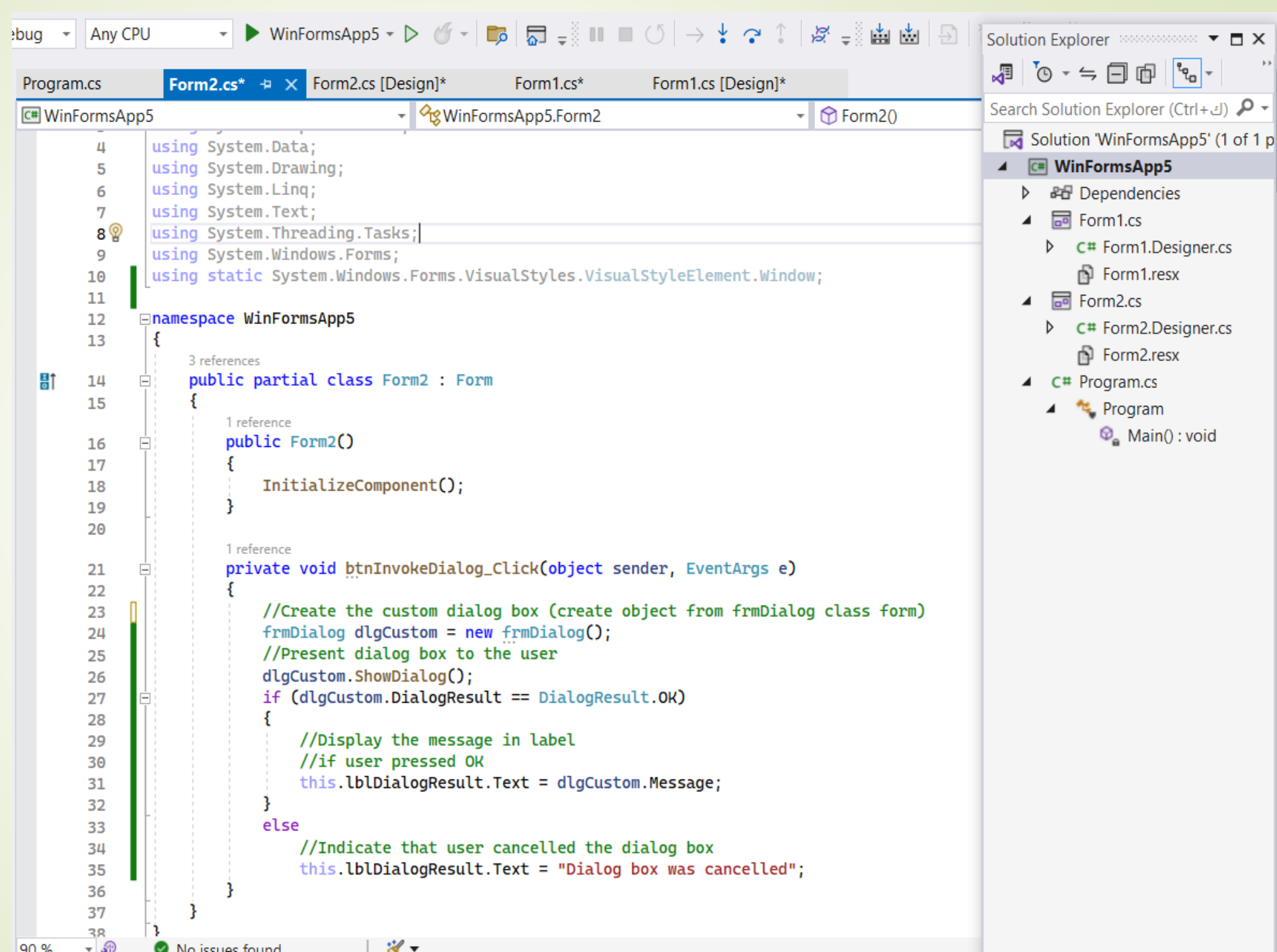
Design the second form





- 
1. Place a Button control and a Label control on the form. Name the Button control **btnInvokeDialog**, and change the Text property to **Invoke Dialog**.
 2. Name the Label control **lblDialogResult**, and change the Text property to **Click the button to invoke a custom dialog box**.

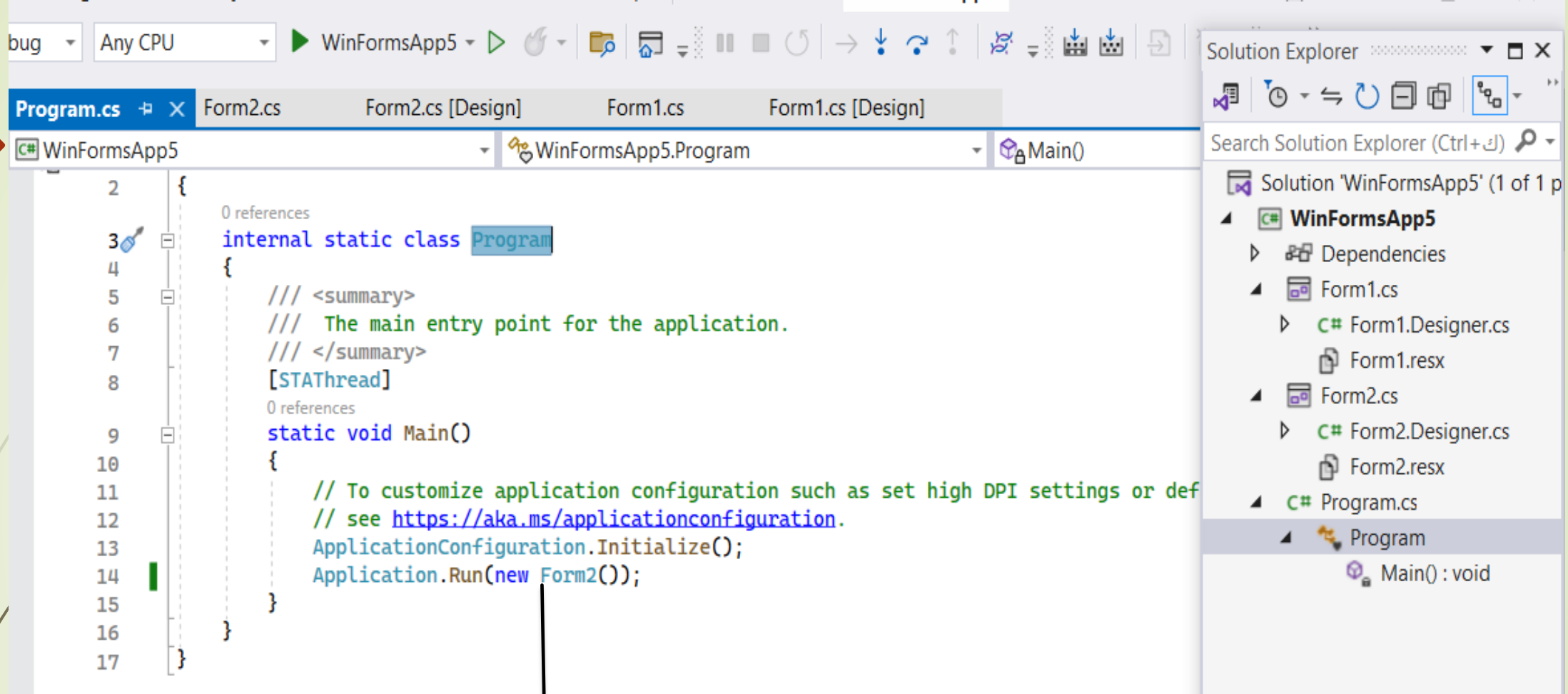
Attach an event handler to the **Click** event of btnInvokeDialog and add the following code to it



The screenshot displays the Visual Studio IDE with the following components:

- Toolbar:** Includes icons for debugging, running, and other development actions.
- Tab Bar:** Shows open files: Program.cs, Form2.cs* (active), Form2.cs [Design]*, Form1.cs*, and Form1.cs [Design]*.
- Code Editor:** Contains the C# code for Form2.cs. The code includes using statements for System.Data, System.Drawing, System.Linq, System.Text, System.Threading.Tasks, and System.Windows.Forms. It defines a namespace WinFormsApp5 and a partial class Form2 : Form. The class has a constructor Form2() and a Click event handler btnInvokeDialog_Click. The event handler creates a custom dialog box (frmDialog), shows it, and updates the text of lblDialogResult based on the dialog result (OK or Cancelled).
- Solution Explorer:** Shows the project structure for WinFormsApp5, including dependencies, Form1.cs, Form1.Designer.cs, Form1.resx, Form2.cs, Form2.Designer.cs, Form2.resx, Program.cs, and Program.Main() : void.

```
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10 using static System.Windows.Forms.VisualStyles.VisualStyleElement.Window;
11
12 namespace WinFormsApp5
13 {
14     3 references
15     public partial class Form2 : Form
16     {
17         1 reference
18         public Form2()
19         {
20             InitializeComponent();
21         }
22
23         1 reference
24         private void btnInvokeDialog_Click(object sender, EventArgs e)
25         {
26             //Create the custom dialog box (create object from frmDialog class form)
27             frmDialog dlgCustom = new frmDialog();
28             //Present dialog box to the user
29             dlgCustom.ShowDialog();
30             if (dlgCustom.DialogResult == DialogResult.OK)
31             {
32                 //Display the message in label
33                 //if user pressed OK
34                 this.lblDialogResult.Text = dlgCustom.Message;
35             }
36             else
37             {
38                 //Indicate that user cancelled the dialog box
39                 this.lblDialogResult.Text = "Dialog box was cancelled";
40             }
41         }
42     }
43 }
```

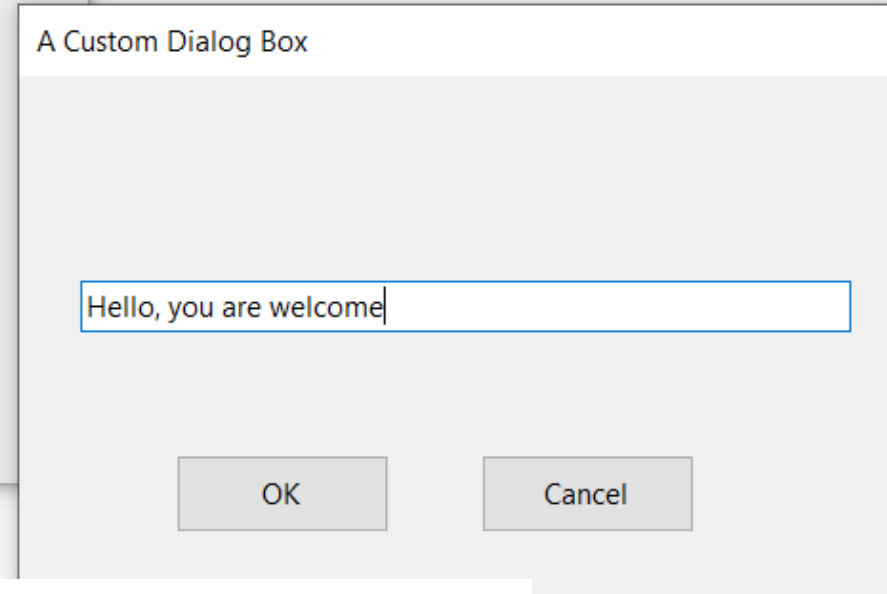
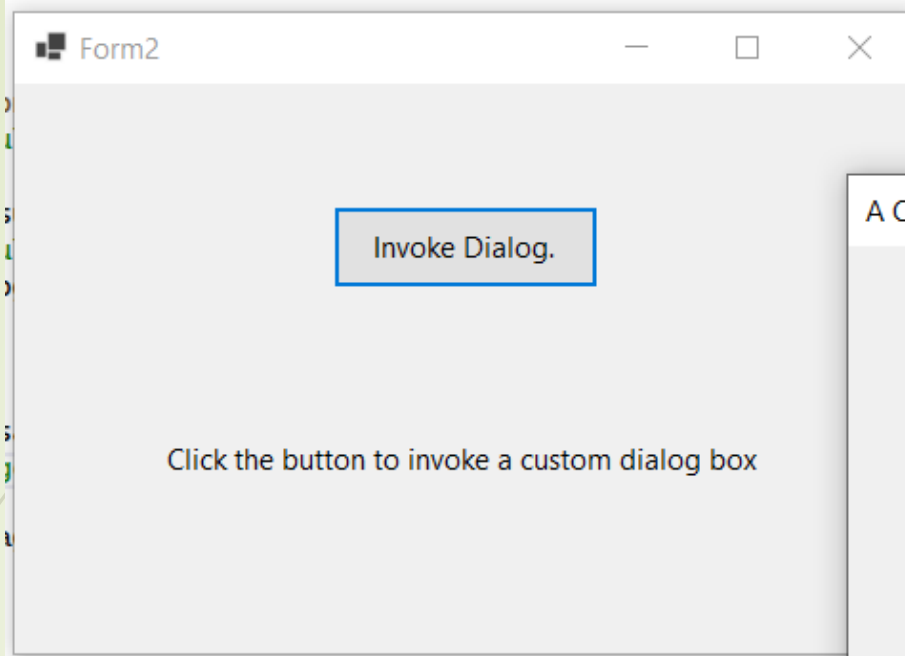


Set the form(form2) as the start up object for the project.

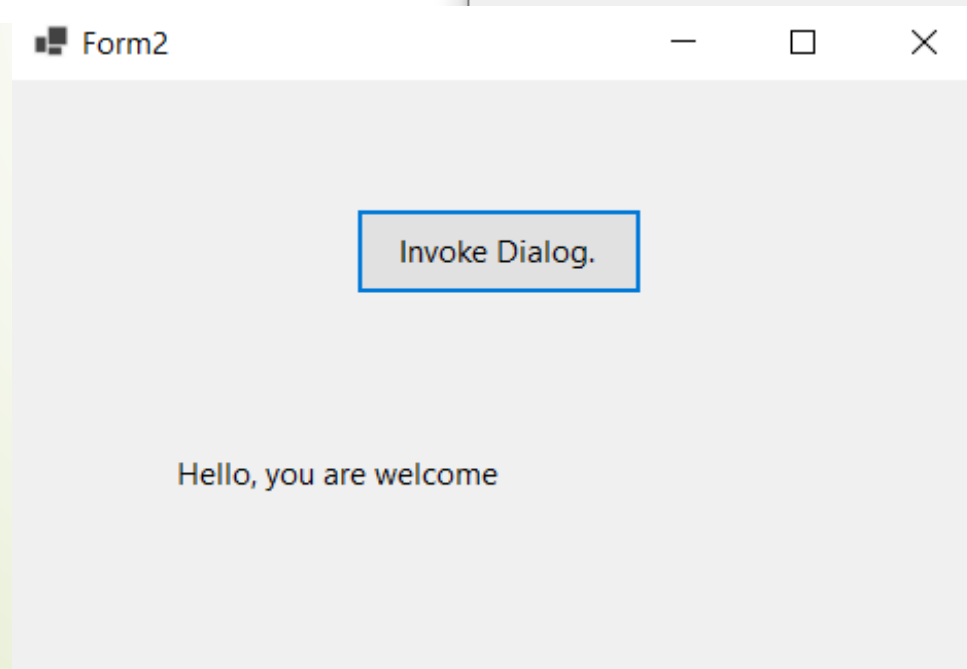
The ShowDialog() method

- The ShowDialog() method displays a form as **a modal dialog box**. All the buttons on the form that need to return results have their **DialogResult** properties set to any of the **DialogResult** enumeration values except **DialogResult.None**.
- When the user **clicks** one of these buttons, the button sets the form's **DialogResult** property with **the DialogResult property of the button** and closes the form automatically after running the Click event handler (if any).

```
btnOK.DialogResult = System.Windows.Forms.DialogResult.OK;  
//make the default value for the btncancel dialog result to cancel  
btnCancel.DialogResult = DialogResult.Cancel;
```



page;





References



Developing and Implementing Windows Based Applications with Microsoft Visual C# .NET and Microsoft Visual Studio.